Phased Implementation Plan & Roadmap

Revision: 1 **Last modified:** 2026-07-24T00:00:00Z

Phase : Foundation (Weeks -)

Goal: Establish the LLM infrastructure, Constitution integration, MCP deployment, and SpecKit installation so all subsequent phases build on a solid foundation.

Week — Environment & LLM Setup

Day	Task	Owner	Deliverable
Mon	Provision Helix LLM API access — obtain API key, endpoint, rate limits	Ops	API credentials in `.env`
Mon	Install `node >=18`, `bash >=5`, `jq`, `ripgrep`, `graphviz` on all hosts	Ops	Verified `command -v` for each tool
Tue	Clone `helix-nano-bridge` template repo, scaffold directory structure	Dev	`tree constitution/submodules/helix-nano-bridge/`
Tue	Create `constitution/submodules/helix-nano-bridge/helix-deps.yaml` per §11.4.28(C)	Dev	Valid YAML, parsed by `constitution/scripts/check_deps.sh`
Wed	Configure Helix LLM fallback chain: `helix-nano → deepseek-v4-pro → opus-xhigh`	Dev	`.env.template` with all 3 endpoints
Wed	Write `bin/llm_probe.sh` — health-check script that tests all 3 LLM backends	Dev	Exit 0 = all backends reachable
Thu	Write `lib/llm_client.sh` — unified client calling Helix API with retry + backoff	Dev	Unit test: 3 consecutive successful calls
Fri	Integration test: `llm_probe.sh` → all backends GREEN, fallback chain verified	QA	Test log in `qa-results/week1/`
Fri	Week 1 retrospective + `docs/CONTINUATION.md` update	All	Updated §3 Active work

Week — Constitution Integration

Day	Task	Owner	Deliverable
Mon	Study `constitution/Constitution.md` §11.4.27 (HelixQA), §11.4.43 (TDD), §11.4.115 (RED/GREEN)	Dev	Notes in `docs/research/constitution_audit.md`
Tue	Map every SpecKit extension hook to the corresponding Constitution §-letter	Dev	Traceability matrix `docs/research/speckit_constitution_map.md`
Tue	Write `CM-HELIX-NANO-BRIDGE-EXTENSION-EXISTS` pre-build gate	Dev	Gate in `pre_build_verification.sh`
Wed	Write paired §1.1 meta-test mutation for the gate	Dev	Mutation in `meta_test_false_positive_proof.sh`
Thu	Wire `helix-nano-bridge` into `docs_chain` per §11.4.106	Dev	`.docs_chain/contexts/helix_nano_bridge.yaml`
Fri	Verify all 5 Constitution `CM-COVENANT-*` propagation gates still GREEN after extension	QA	`pre_build_verification.sh` exit 0, 0 WARN
Fri	Week 2 retrospective	All	CONTINUATION.md updated

Week — MCP Server Deployment

Day	Task	Owner	Deliverable
Mon	Scaffold `superbridge-mcp` TypeScript project: `package.json`, `tsconfig.json`, `src/index.ts`	Dev	`npm install && npm run build` succeeds
Tue	Implement `execute_tdd_cycle` tool with RED→GREEN→VERIFY pipeline	Dev	Integration test: mocked TDD cycle exits correctly
Wed	Implement `verify_anti_bluff` tool — evidence file parsing + pattern matching	Dev	Unit tests for golden-good / golden-bad inputs
Thu	Implement `nano_task_decompose` + `nano_task_execute` + `nano_task_graph` tools	Dev	5 tools registered, each with input validation
Fri	Wire MCP server into Claude Code, Codex, Cursor, Gemini CLI configs	Dev	`mcp_server_test.sh` probes each agent
Fri	Week 3 retrospective	All	CONTINUATION.md updated

Week — SpecKit Installation & Smoke Test

Day	Task	Owner	Deliverable
Mon	Install SpecKit CLI: `specify --version` resolves	Dev	`specify --version` ≥ 2.0.0
Tue	Install SpecKit extensions: `superb`, `helix-nano-bridge` (skeleton)	Dev	`specify extension list` shows both
Tue	Create skeleton `extension.yml` with all 5 commands registered	Dev	`specify helix-nano-bridge --help` prints usage
Wed	Write `speckit.helix-nano-bridge.decompose.md` command spec	Dev	Markdown file with full spec per EXTENSION_DEVELOPMENT.md
Thu	Write remaining 4 command specs: `execute.md`, `graph.md`, `status.md`, `retry.md`	Dev	All 5 command specs complete
Fri	End-to-end smoke test: `specify feature new test-feature` → decompose → execute (dry-run)	QA	`qa-results/week4/smoke_test.log`
Fri	Phase 1 milestone review — foundation DECLARED READY	All	`CM-HELIX-NANO-BRIDGE-EXTENSION-EXISTS` GREEN

Phase	Deliverables

```
constitution/submodules/helix-nano-bridge/
├── bin/
│   ├── llm_probe.sh                    # LLM health check
│   └── helix-nano-bridge               # Main entry point (skeleton)
├── lib/
│   └── llm_client.sh                   # Unified LLM client
├── spec/
│   └── helix-nano-bridge/
│       ├── extension.yml               # SpecKit extension manifest
│       ├── speckit.helix-nano-bridge.decompose.md
│       ├── speckit.helix-nano-bridge.execute.md
│       ├── speckit.helix-nano-bridge.graph.md
│       ├── speckit.helix-nano-bridge.status.md
│       └── speckit.helix-nano-bridge.retry.md
├── mcp/
│   └── superbridge-mcp/
│       ├── package.json
│       ├── tsconfig.json
│       ├── src/index.ts                # 5-tool MCP server
│       └── dist/index.js               # Built artifact
├── helix-deps.yaml                     # §11.4.28(C) dependency spec
└── .env.template
```

2 5 8

Phase : Nano-Task Engine (Weeks –)

Goal: Build the nano-task decomposition engine, DAG dependency resolver, TDD execution pipeline, and anti-bluff oracle — the core intellectual property.

Week — Nano-Task Schema & Decompose Command

Day	Task	Owner	Deliverable
Mon	Define `nano-tasks.json` JSON Schema (JSON Schema draft-2020-12)	Dev	`spec/nano-tasks.schema.json` — validates with `ajv`
Tue	Implement `nano_decompose_task()` in `lib/nano_bridge_common.sh`	Dev	Parses tasks.md → extracts files/symbols
Wed	Implement `nano_build_dependency_dag()` — adjacency list from plan.md	Dev	DAG builder with test coverage
Thu	Implement `nano_detect_cycles()` — Kahn's algorithm with DFS fallback	Dev	Cycle detection test: inject cycle → detected
Fri	Wire `helix-nano-bridge decompose` command end-to-end	Dev	`nano-tasks.json` produced from real `tasks.md`
Fri	Week 5 retrospective	All	CONTINUATION.md updated

Week — DAG Engine & Phase Assignment

Day	Task	Owner	Deliverable
Mon	Implement `nano_assign_phases()` — level-order grouping	Dev	Tasks assigned to phases, no file collisions in same phase
Tue	Implement critical path computation — longest path in DAG	Dev	`nano-tasks.json` includes `critical_path_length`
Wed	Implement `nano_generate_mermaid()` / `nano_generate_dot()`	Dev	Mermaid and DOT output renderable in tools
Thu	Wire `helix-nano-bridge graph` command	Dev	`--format mermaid` → valid chart, `--format svg` → renders
Fri	Test DAG correctness on 10 sample `tasks.md` files (varying complexity)	QA	All 10 produce acyclic, correctly-phased DAGs
Fri	Week 6 retrospective	All	CONTINUATION.md updated

Week — TDD Execution Engine

Day	Task	Owner	Deliverable
Mon	Implement `nano_execute_single_task()` — RED→GREEN→VERIFY cycle	Dev	Core execution loop
Tue	Implement RED phase: run test, assert FAIL, capture evidence	Dev	`RED_<timestamp>.log` with exit code ≠ 0
Wed	Implement GREEN phase: run test, assert PASS, 3-iteration consistency	Dev	`GREEN_<timestamp>.log` with exit code = 0, all 3 identical
Thu	Implement VERIFY phase: anti-bluff oracle checks evidence files	Dev	`verdict.json` with PASS/FAIL + rationale
Fri	Wire `helix-nano-bridge execute` command — phase-by-phase execution	Dev	Phases execute in order, parallel within phase
Fri	Week 7 retrospective	All	CONTINUATION.md updated

Week — Unit Tests & Anti-Bluff Oracle

Day	Task	Owner	Deliverable
Mon	Write unit tests for `nano_decompose_task()` — 10+ test cases	Dev	100% pass rate
Tue	Write unit tests for `nano_build_dependency_dag()` — cycle/no-cycle/multi-dep	Dev	100% pass rate
Wed	Write unit tests for `nano_execute_single_task()` — RED-fail/GREEN-pass/verify	Dev	100% pass rate
Thu	Implement anti-bluff oracle golden-good / golden-bad fixtures	Dev	Golden-good PASSES oracle, golden-bad FAILS it
Fri	Wire paired §1.1 mutation for the execution gate	Dev	Stripping gate check → gate FAILs
Fri	Phase 2 milestone review — nano-task engine DECLARED READY	All	`CM-NANO-TASK-ENGINE` GREEN

Phase Deliverables

```
lib/
├── nano_bridge_common.sh              # Full implementation (~600 lines)
├── anti_bluff_oracle.sh               # Evidence validation oracle
├── test_nano_bridge.sh                # Unit test suite for library
└── test_anti_bluff_oracle.sh          # Oracle golden-good/golden-bad
tests

bin/
├── helix-nano-bridge-decompose        # Full implementation
├── helix-nano-bridge-execute          # Full implementation
├── helix-nano-bridge-graph            # Full implementation
├── helix-nano-bridge-status           # Full implementation
└── helix-nano-bridge-retry            # Full implementation

spec/
└── nano-tasks.schema.json             # JSON Schema for nano-tasks.json

qa-results/week8/
├── unit_tests.log                     # All unit tests GREEN
├── decompose_10_samples.log           # 10/10 DAGs correct
└── oracle_golden_fixtures.log         # Good=PASS, Bad=FAIL
```

3

9 12

Phase : Superbridge Integration (Weeks –)

Goal: Build the MCP server, SuperB extension integration, and E2E tests so the full pipeline works from `tasks.md` to verified implementation.

Week — MCP Server Completion

Day	Task	Owner	Deliverable
Mon	Complete `execute_tdd_cycle` tool — full RED→GREEN→VERIFY with evidence capture	Dev	Tool returns structured PASS/FAIL verdict
Tue	Complete `verify_anti_bluff` tool — expected/forbidden pattern matching	Dev	Tool catches golden-bad inputs
Wed	Complete `nano_task_decompose` tool — calls bash library via `exec`	Dev	Tool produces valid `nano-tasks.json`
Thu	Complete `nano_task_execute` + `nano_task_graph` tools	Dev	All 5 tools functional
Fri	MCP server integration test: Claude Code calls `execute_tdd_cycle` on a mock task	QA	PASS verdict with evidence JSON
Fri	Week 9 retrospective	All	CONTINUATION.md updated

Week — SuperB Integration

Day	Task	Owner	Deliverable
Mon	Map `superb` 7-command workflow to `helix-nano-bridge` 5 commands	Dev	`superb/integrations/helix-nano-bridge.yaml`
Tue	Wire `superb:plan` → `decompose`, `superb:implement` → `execute`	Dev	SpecKit hooks fire correctly
Wed	Implement `implementation-gate.js` — 7 checks before execution	Dev	`node implementation-gate.js <feature_dir>`
Thu	Wire `superb:verify` → `status --format report --evidence`	Dev	SuperB verify phase shows nano-task status
Fri	Wire `superb:ship` gated on 100% GREEN nano-tasks	Dev	Ship blocked when any task FAIL
Fri	Week 10 retrospective	All	CONTINUATION.md updated

Week — E E Tests

Day	Task	Owner	Deliverable
Mon	Create test feature: `specify feature new e2e-nano-test` with 3 tasks	Dev	`tasks.md` with 3 tasks, 5 dependencies
Tue	Run full pipeline: `tasks.md` → decompose → execute → graph → status	QA	100% GREEN, evidence captured for all nano-tasks
Wed	Inject a bug: modify a RED test to pass when it should fail → verify gate BLOCKS	QA	Gate exit 2, `FAIL` verdict recorded
Thu	Test retry: force-fail 2 tasks → `retry` → tasks recover and go GREEN	QA	Retry log shows RED→GREEN→VERIFY cycles
Fri	Test parallel execution: 24 independent nano-tasks → all run concurrently within phase	QA	Phase execution time ≤ sum of longest 2 tasks
Fri	Week 11 retrospective	All	CONTINUATION.md updated

Week — Completion & Polish

Day	Task	Owner	Deliverable
Mon	Write user documentation: `docs/guides/HELIX_NANO_BRIDGE_USER_GUIDE.md`	Dev	Complete user guide, exported to HTML+PDF+DOCX
Tue	Write installation guide: `docs/guides/HELIX_NANO_BRIDGE_INSTALL.md`	Dev	Step-by-step install, verified on fresh clone
Wed	Polish error messages — every exit code has a human-readable explanation	Dev	`helix-nano-bridge --help` shows all exit codes
Thu	Add `helix-nano-bridge` to `constitution/scripts/post_update_hook.sh` per §11.4.164	Dev	Auto-registered on constitution pull
Fri	Phase 3 milestone review — Superbridge integration DECLARED READY	All	Full E2E test log GREEN
Fri	Week 12 retrospective	All	CONTINUATION.md updated

Phase Deliverables

```
mcp/superbridge-mcp/
├── src/
│   ├── index.ts                    # 5-tool MCP server (complete)
│   ├── tdd_cycle.ts                # execute_tdd_cycle implementation
│   ├── anti_bluff.ts               # verify_anti_bluff implementation
│   ├── decompose.ts                # nano_task_decompose
implementation
│   ├── execute.ts                  # nano_task_execute implementation
│   └── graph.ts                    # nano_task_graph implementation
├── tests/
│   ├── tdd_cycle.test.ts
│   ├── anti_bluff.test.ts
│   ├── decompose.test.ts
│   └── e2e.test.ts
├── dist/                           # Built JS output
└── package.json

superb/integrations/
└── helix-nano-bridge.yaml          # SuperB 7-command wiring

bin/
└── implementation-gate.js          # Pre-execution gate (7 checks)

docs/guides/
├── HELIX_NANO_BRIDGE_USER_GUIDE.md
└── HELIX_NANO_BRIDGE_INSTALL.md

qa-results/week11/
├── e2e_full_pipeline.log           # GREEN
├── gate_blocks_bad_test.log        # Gate caught the injected bug
├── retry_recovery.log              # Tasks recovered
└── parallel_benchmark.log          # Performance data
```

4

1 3 1 6

Phase : Production Hardening (Weeks -)

Goal: Multi-host deployment, 100% test coverage, performance benchmarking, security audit, and documentation finalization — ready for production use.

Week — Multi-Host RPC & Scalability

Day	Task	Owner	Deliverable
Mon	Implement `superbridge-mcp` over HTTP/SSE transport (beyond stdio)	Dev	MCP server listens on TCP port, SSE streaming
Tue	Implement remote execution: nano-tasks dispatched to worker nodes	Dev	Task dispatched via `POST /execute` to remote MCP server
Wed	Implement distributed evidence store: shared `qa-results/` over NFS/SSHFS	Dev	All hosts read/write same evidence directory
Thu	Implement leader election for phase coordination — only one host advances phases	Dev	Leader lock via `flock` on shared NFS
Fri	Load test: 4 worker nodes, 200 nano-tasks, measure throughput	QA	Throughput report in `qa-results/week13/parallel_benchmark.md`
Fri	Week 13 retrospective	All	CONTINUATION.md updated

Week — % Test Coverage

Day	Task	Owner	Deliverable
Mon	Audit current test coverage — identify gaps	QA	Coverage report: `coverage_report.json`
Tue	Write tests for all error paths in `nano_bridge_common.sh`	Dev	Every `exit 1/2/3/4` path has a test
Wed	Write tests for all 5 MCP tools with edge cases	Dev	`npm test` → 100% pass, 80%+ coverage
Thu	Write chaos tests per §11.4.85: process death, disk full, OOM injection	Dev	Chaos tests in `tests/chaos/`
Fri	Write stress tests per §11.4.85: 100 concurrent decomposes, 1000-task DAG	QA	Stress report in `qa-results/week14/stress_tests.md`
Fri	Week 14 retrospective	All	CONTINUATION.md updated

Week — Performance Benchmarking & Security Audit

Day	Task	Owner	Deliverable
Mon	Benchmark decompose: 50-task `tasks.md` → nano-tasks (target: <5s)	QA	Latency p50/p95/p99 in benchmark report
Tue	Benchmark execute: 50 nano-tasks sequential (target: <2 min)	QA	Per-task latency distribution
Wed	Benchmark graph: 500-node DAG → Mermaid/DOT/SVG (target: <1s)	QA	Format-specific rendering times
Thu	Security audit: MCP server input validation, injection vectors, credential handling	Sec	Audit report in `docs/audit/helix_nano_bridge_security.md`
Fri	Fix all HIGH/MEDIUM security findings	Dev	Re-audit: 0 HIGH, 0 MEDIUM findings
Fri	Week 15 retrospective	All	CONTINUATION.md updated

Week — Documentation & Release

Day	Task	Owner	Deliverable
Mon	Finalize `EXTENSION_DEVELOPMENT.md` — ensure every section complete	Dev	Document passes `CM-DOC-REVISION-HEADER-PRESENT`
Tue	Finalize `IMPLEMENTATION_PLAN.md` — verify all weeks mapped to actual deliverables	Dev	Document passes `CM-DOC-REVISION-HEADER-PRESENT`
Wed	Generate all 4 export formats (MD+HTML+PDF+DOCX) per §11.4.65	Dev	All mtimes ≥ source `.md`
Thu	Run full Constitution compliance sweep — all `CM-*` gates GREEN	QA	`pre_build_verification.sh` exit 0, 0 FAIL, 0 WARN
Fri	Tag `constitution` Submodule + create release `helix-nano-bridge-v1.0.0`	Ops	Tag on all remotes, `pack.sh` archive published
Fri	Phase 4 milestone review — PRODUCTION READY declared	All	All 4 phases complete

Phase Deliverables

```
mcp/superbridge-mcp/
├── src/
│   └── transport/
│       ├── stdio.ts                    # Existing stdio transport
│       └── http_sse.ts                 # New HTTP/SSE transport
├── tests/
│   ├── chaos/
│   │   ├── process_death.test.ts
│   │   ├── disk_full.test.ts
│   │   └── oom_injection.test.ts
│   └── stress/
│       ├── concurrent_decompose.test.ts
│       └── large_dag.test.ts
└── coverage/                           # Istanbul coverage reports

docs/
├── audit/
│   └── helix_nano_bridge_security.md   # Security audit findings
├── benchmarks/
│   └── performance_2026Q3.md           # Latency + throughput data
└── changelogs/
    └── v1.0.0.md                       # Release changelog

dist/
└── helix-nano-bridge-1.0.0.tar.gz      # Packaged release
```

Milestones & Deliverables

Milestone	End of Phase	Deliverables	Evidence
M1: Foundation	Week 4	LLM configured, SpecKit installed, MCP skeleton running, 5 command specs complete	`llm_probe.sh` GREEN, `specify helix-nano-bridge --help` works, MCP server connects to Claude Code
M2: Nano-Task Engine	Week 8	Decompose, execute, graph, status, retry all functional; DAG engine correct; anti-bluff oracle validated	All 5 commands tested on real `tasks.md`, `nano-tasks.json` valid, golden-good/bad fixtures pass
M3: Superbridge Integration	Week 12	MCP server complete (5 tools), SuperB integrated, E2E pipeline GREEN, user docs published	Full E2E log: decompose → execute → graph → status all GREEN
M4: Production Ready	Week 16	Multi-host, 100% coverage, security audited, performance benchmarked, documented, tagged	`pre_build_verification.sh` 0 FAIL 0 WARN, release tag pushed, `pack.sh` archive with SHA256

Risk Register

#	Risk	Probability	Impact	Mitigation	Owner
R1	Helix LLM API rate-limits during decompose phase	Medium	High	Fallback to deepseek-v4-pro; cache decompositions; batch LLM requests	Dev Lead
R2	Dependency cycles in complex `tasks.md` files	Medium	Medium	Kahn's algorithm + DFS fallback in `nano_detect_cycles()`; operator-override flag	Dev
R3	MCP server stdio transport incompatible with some agents	Medium	High	Ship both stdio and HTTP/SSE transports; test on all 4 agents in Week 3	Dev
R4	Evidence directory fills disk during long execution runs	Low	High	Per-task evidence size budget; auto-prune on GREEN; disk-space gate before execution	Ops
R5	Phase assignment creates false sharing (two tasks modify same file)	Medium	Medium	Phase collision detection in `nano_assign_phases()`; `implementation-gate.js` check #7	Dev
R6	Constitution amendment invalidates SpecKit hook mapping	Low	High	`CM-HELIX-NANO-BRIDGE-EXTENSION-EXISTS` gate catches drift; `constitution/scripts/post_update_hook.sh` auto-revalidates	Dev Lead

#	Risk	Probability	Impact	Mitigation	Owner
R7	Threadripper NUMA topology causes uneven task distribution	Medium	Low	`numactl --cpunodebind` per-worker; hardware detection auto-configures	Dev
R8	SpecKit API breaks backward compatibility (v3.x)	Low	Medium	Extension manifest declares `specKitVersion: ">=2.0.0"`; pin version in CI	Dev
R9	Security vulnerability in MCP server input handling	Medium	Critical	Input validation on all 5 tools; security audit Phase 4 Week 15; never eval user input	Sec
R10	Operator time unavailable for milestones (blocked decisions)	Medium	High	Per §11.4.66 interactive clarification; autonomous decisions on low-blast-radius items per §11.4.101	PM

Resource Requirements

Hardware (per development host)

Component	Minimum	Recommended	Purpose
CPU	16 cores	32+ cores (Threadripper)	Parallel nano-task execution
RAM	32 GB	64 GB	LLM context windows + parallel workers
GPU	None	NVIDIA RTX 4090 / A6000	LLM inference offload
Storage	50 GB free	200 GB NVMe	Evidence artifacts, build artifacts
Network	1 Gbps	10 Gbps	Multi-host coordination, evidence sync

Software

Tool	Version	Purpose
Node.js	≥18 LTS	MCP server runtime
Bash	≥5.0	Core scripting
jq	≥1.6	JSON processing
ripgrep	≥13	Fast file search
Graphviz	≥2.50	DAG rendering (DOT/SVG/PNG)
TypeScript	≥5.0	MCP server source
SpecKit CLI	≥2.0.0	Extension host
Superpowers CLI	≥3.1.0	Skill host

Personnel

Role	Count	Commitment
Lead Developer	1	Full-time, 16 weeks
Developer (MCP + JS)	1	Full-time, weeks 3–15
QA Engineer	1	50%, weeks 4–16
Security Auditor	1	2 weeks, weeks 15
Operator/PM	1	Advisory, milestone gates

Success Criteria (Anti-Bluff Evidence)

Each phase is complete ONLY when the following evidence is captured:

Phase — Success Criteria

#	Criterion	Evidence Type
P1.1	`llm_probe.sh` exits 0 against all 3 LLM backends	Captured stdout with per-backend latency
P1.2	`specify helix-nano-bridge --help` prints usage for all 5 commands	Captured terminal output
P1.3	MCP server connects to Claude Code — `superbridge-mcp` listed in `ListTools`	MCP handshake log
P1.4	`CM-HELIX-NANO-BRIDGE-EX-TENSION-EXISTS` gate GREEN	`pre_build_verification.sh` log
P1.5	Paired §1.1 mutation FAILs the gate	`meta_test_false_positive_proof.sh` log

Phase — Success Criteria

#	Criterion	Evidence Type
P2.1	`decompose` produces valid `nano-tasks.json` from 10 diverse `tasks.md` files	10 `nano-tasks.json` files, all passing `ajv` validation
P2.2	Dependency graph is acyclic for all 10 samples	Cycle detection: 0 cycles found
P2.3	`execute` runs a full TDD cycle: RED fails → GREEN passes → VERIFY confirms	`verdict.json` with `PASS` and 3 iterations
P2.4	Anti-bluff oracle: golden-good → PASS, golden-bad → FAIL	Oracle test log
P2.5	All unit tests GREEN for library functions	Test runner output, 0 failures

Phase Success Criteria

#	Criterion	Evidence Type
P3.1	Full E2E: `tasks.md` (3 tasks) → decompose → execute → all GREEN	`qa-results/e2e/full_pipeline.log`
P3.2	`implementation-gate.js` BLOCKS execution when RED test is missing	Gate exit 2, `FAIL` verdict
P3.3	Retry recovers intentionally-failed tasks	Retry log: RED→GREEN→VERIFY for each recovered task
P3.4	SuperB `superb:ship` blocked when nano-tasks not all GREEN	Ship gate log
P3.5	User guide complete: HTML+PDF+DOCX exports pass `CM-DOCS-EXPORT-SYNC`	Export freshness gate GREEN

Phase Success Criteria

#	Criterion	Evidence Type
P4.1	4 worker nodes execute 200 nano-tasks in parallel, throughput ≥ 10 tasks/s	Benchmark report in `qa-results/benchmarks/`
P4.2	Test coverage ≥ 85% line, ≥ 80% branch	Istanbul/NYC coverage report
P4.3	Chaos tests: process death, disk full, OOM → graceful degradation (no crash)	Chaos test logs with recovery traces
P4.4	Security audit: 0 HIGH, 0 MEDIUM findings	`docs/audit/helix_nano_bridge_security.md`
P4.5	`pre_build_verification.sh` exit 0, 0 FAIL, 0 WARN on release tag	Full pre-build sweep log
P4.6	Release tag `helix-nano-bridge-v1.0.0` pushed to all remotes per §11.4.113	`git ls-remote --tags` shows tag on all remotes
P4.7	`pack.sh` archive SHA256 published and verified	Release manifest `release-1.0.0.json`